

TNA RAG and LLM Implementation Guide

Training Needs Analysis System

Implementation overview, request workflow, security boundaries, APIs, and current limitations

Generated: 8 September 2026

1. Executive Summary

The system uses Retrieval-Augmented Generation (RAG) to find approved evidence from training documents and uses a local Mistral language model to summarize or explain that evidence.

Core principle: RAG decides which approved content may be used. Mistral writes the response. Django and PostgreSQL remain authoritative for permissions, approvals, users, costs, statuses, and official business data.

2. Architecture

```
Employee uploads PDF
      |
      v
Django Attachment and KnowledgeDocument
      |
      v
pypdf extracts text page by page
      |
      v
Text is split into overlapping chunks
      |
      v
nomic-embed-text creates 768-dimensional vectors
      |
```

```

      v
PostgreSQL + pgvector stores chunks and embeddings
      |
      v
HOD approves the request and document
      |
      v
Approved chunks become searchable
      |
      v
User asks a question
      |
      v
Question embedding retrieves nearest approved chunks
      |
      v
Mistral receives the question plus retrieved evidence
      |
      v
Grounded answer with source filename and page

```

3. Document Ingestion

Implemented in `backend/TNAbackend/ai/services.py` through `ingest_attachment()`.

1. The uploaded file is stored as an `Attachment`.
2. A related `KnowledgeDocument` is created.
3. PDF pages are read with `pypdf`.
4. Text is split into 1,200-character chunks with 200-character overlap.
5. Each chunk is embedded by `nomic-embed-text`.
6. Chunks store text, page number, embedding, and embedding model.
7. The document is marked `INDEXED`, `FAILED`, or `UNSUPPORTED`.

PDF extraction is implemented. Other file types are currently marked unsupported by the first RAG slice.

4. Approval and Search Eligibility

```

ATTACHMENT_WAIT
-> employee uploads document
PENDING_DEPT
-> HOD submits/approves
SUBMITTED

```

-> Managing Director approves
APPROVED

A document is searchable only when:

- `extraction_status = INDEXED`
- `KnowledgeDocument.approval_status = APPROVED`
- `Attachment.approval_status = APPROVED`

Pending, rejected, failed, and unapproved documents are excluded before retrieval.

5. Authorization

The `authorized_documents()` function filters evidence before the language model sees it.

User	Accessible approved documents
Administrator, HR, Trainer, Director	Approved organizational knowledge available to their role
Department Head	Their own request documents and documents belonging to supervised employees
Employee or participant	Approved documents connected to requests they own or participate in

A request-specific query can include a request ID, which limits retrieval to that request.

6. Retrieval

Endpoints:

- `POST /api/v1/ai/knowledge/query/` retrieves approved evidence only.
- `POST /api/v1/ai/knowledge/answer/` retrieves evidence and generates an answer.

The question is embedded with `nomic-embed-text`. PostgreSQL pgvector orders matching chunks by cosine distance. If embeddings are unavailable, a text-term fallback is used.

7. Mistral LLM Generation

The service `generate_grounded_answer()` sends the question and selected evidence to the local Mistral model.

Default local endpoint:

```
POST http://127.0.0.1:11434/api/chat
Model: mistral
```

The prompt instructs the model to:

- Answer only from supplied approved evidence.
- Never invent unsupported facts.
- Never approve requests or bypass permissions.
- Never calculate authoritative budgets.
- State when evidence is insufficient.
- Use source citations.

8. Answer Response

```
{
  "status": "COMPLETED",
  "answer": "Grounded answer...",
  "citations": ["chunk-id"],
  "sources": [
    {
      "file_name": "training-manual.pdf",
      "page_number": 3,
      "text": "Approved source passage..."
    }
  ],
  "error": ""
}
```

Possible response states are `COMPLETED` , `FAILED` , and `INSUFFICIENT_EVIDENCE` .

9. Training Request Overview

When the HOD submits a request:

1. Request attachments become approved knowledge.
2. RAG retrieves the request evidence.
3. Mistral generates a grounded learning overview.
4. The overview is saved in `TrainingLearningOverview` .
5. The creator, participants, and next approver receive notifications.
6. The request moves to `SUBMITTED` for the Managing Director.

Endpoint:

```
GET /api/v1/tna/requests/<id>/learning/
```

10. Daily Learning Questions

Daily questions are stored in `TrainingDailyQuestion` and are grounded in approved request documents.

- Questions begin on the request `start_date`.
- Questions stop after `end_date`.
- Only one question is stored per request per date.
- The current question is created when the learning endpoint is opened.

Current limitation: daily questions are generated lazily when requested. A background scheduler for midnight generation and daily push notifications is not yet implemented.

11. Frontend

The user-facing assistant is implemented in `tna-frontend/src/pages/KnowledgeAssistant.jsx`.

- Users ask questions about approved documents.
- Answers show grounded status.
- Source filename and page are displayed.
- Source excerpts are visible.
- Technical model names are hidden from normal users.

Request tracking displays the learning overview and current daily question to the creator and participants.

12. Audit and Traceability

The system records:

Record	Purpose
<code>AIQueryAudit</code>	User query, request, source IDs, and result count
<code>AIRequest</code>	Operation, model, prompt version, source IDs, and payload
<code>AIResponse</code>	Status, answer, citations, latency, and errors

13. Safety Boundaries

RAG and the LLM are used for retrieval, summaries, explanations, overviews, and learning questions only.

Django and PostgreSQL control:

- Authentication and authorization
- Request status and approvals
- Participant membership
- Costs and budgets

- Competency scores
- Compliance records
- Official workflow decisions

14. Main Implementation Files

- `backend/TNAbackend/ai/services.py`
- `backend/TNAbackend/ai/models.py`
- `backend/TNAbackend/ai/views.py`
- `backend/TNAbackend/tna/views.py`
- `backend/TNAbackend/tna/models.py`
- `tna-frontend/src/pages/KnowledgeAssistant.jsx`
- `tna-frontend/src/pages/tna/TrainingRequests.jsx`
- `docs/RAG_IMPLEMENTATION_STATUS.md`

RAG retrieves approved evidence. The LLM generates grounded language from that evidence. The backend remains the authority for system behavior.